Dining Philosopher.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

# define N 5
# define MAX-ROUNDS 3

sem_t chopstick[N];
sem_t room;

void think(int i) {
    printf("Philosopher %d is thinking.\n", i);
    sleep(1);
}

void eat(int i) {
    printf("Philosopher %d is eating.\n", i);
    sleep(1);
}

void* philosopher(void* num) {
    int i = *(int*) num;
    for(int j=0; j<MAX-ROUNDS; j++) {
        think(i);
        sem_wait(&room);
        sem_wait(&chopstick[(i+1)%N]);
        printf("Philosopher %d picked up right fork %d.\n", i, (i+1)%
        sem_wait(&chopstick[i]);
        printf("Philosopher %d picked up left fork %d.\n", i, i);
        eat(i);
        sem_post(&chopstick[i]);
        sem_post(&chopstick[(i+1)%N]);
```

```
        printf("Philosopher %d put down forks %d & %d.\n",
                i, i, (i+1) % N);
        sem_post(&room);
    }
    return NULL;
}
int main() {
    pthread_t phil[N];
    int phil_num[N];
    for(int i=0; i<N; i++) {
        sem_init(&chopstick[i], 0, 1);
        phil_num[i]=i;
    }
    sem_init(&room, 0, N-1);
    for(int i=0; i<N; i++) {
        pthread_create(&phil[i], NULL, philosopher, &phil_num[i]);
    }
    for(int i=0; i<N; i++) {
        pthread_join(phil[i], NULL);
    }
    return 0;
}
```

Output:

Philosopher 2 is thinking
Philosopher 0 is thinking.
Philosopher 3 is thinking.
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 0 picked up right fork 1
Philosopher 2 picked up right fork 3
Philosopher 2 picked up left fork 2
Philosopher 2 is eating.
Philosopher 4 picked up right fork 0
Philosopher 3 picked up right fork 4.
Philosopher 2 put down forks 2 & 3
Philosopher 2 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 3 put down forks 3 and 4
Philosopher 3 is thinking.
" " 2 picked up right fork 3.
" " 4 picked up left fork 4.
" " 4 is eating.
" " 4 put down forks 4 and 0
" " 4 is thinking
" " 0 picked up left fork 0
" " 0 is eating.
" " 3 picked up right fork 4
" " 0 put down 0 and 1.
" " 1 picked up left fork 1.
" " 1 is eating
" " 0 0 is thinking
" " 4 picked up right fork 0
" " 1 put down fork 1 and 2
" " 1 is thinking.
" " 2 picked up left fork 2
" " 2 is eating

Write C program to simulate:

Producer - Consumer Problem using Semaphores. [Bounded Buffer]

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define MAX_ITEMS 15

int buffer[BUFFER_SIZE];
int in=0;
int out=0;

sem_t empty;
sem_t full;
sem_t mutex;
int produce_item(int i){
    return i;
}

void consume_item(int item, int index){
    printf("Consumed: %d from buffer[%d]\n", item, index);
}
void *producer(void *arg){
    int next_produced;
    for(int i=0; i<MAX_ITEMS; i++){
        next_produced= produce_item(i);
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = next_produced;
```

```
        printf("Produced : %d at buffer[%d]\n", next_produced, in);
        in = (in+1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}

void* consumer(void* arg) {
    int next_consumed;
    for (int i=0; i< MAX_ITEMS; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        next_consumed = buffer[out];
        int index = out;
        out = (out+1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        consume_item(next_consumed, index);
        usleep(150000);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
```

```
pthread_join(prod, NULL);
pthread_join(cons, NULL);
sem_destroy(&empty);
sem_destroy(&full);
sem_destroy(&mutex);
return 0;
}
```

Output:

Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Produced: 4 at buffer[4]
Consumed: 3 from buffer[3]
Produced: 5 at buffer[0]
Consumed: 4 from buffer[4]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 5 from buffer[0]
Produced: 8 at buffer[3]
Consumed: 6 from buffer[1]
Produced: 9 at buffer[4]
Produced: 10 at buffer[0]
Consumed: 7 from buffer[2]
Produced: 11 at buffer[1]
Consumed: 8 from buffer[3]
Produced: 12 at buffer[2]
Consumed: 9 from buffer[4]